

Nexa: A Context-Aware Assistant for Service Recommendation on Smartphones

Li Chen
Tsinghua University
Beijing, China
lizchan0426@gmail.com

Kaige Yang
Central Academy of Fine Arts
Beijing, China
kaiiigeyang@gmail.com

Zixuan Gong
Central Academy of Fine Arts
Beijing, China
15968791089@163.com

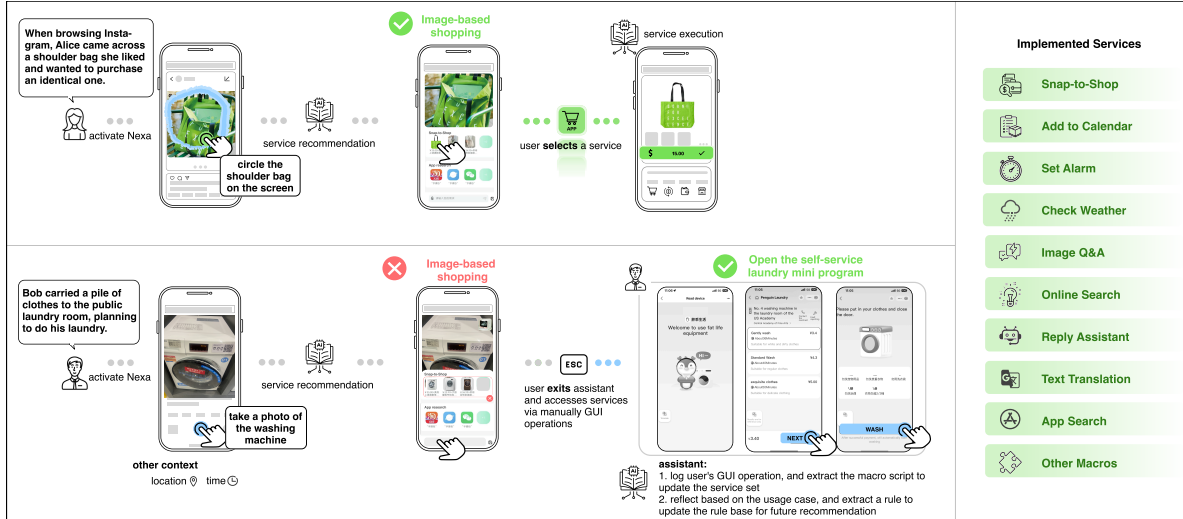


Figure 1: Example scenarios of Nexa. The top shows a correct example: Alice circles a shoulder bag on the screen, and Nexa recommends services based on the circle content. When Alice selects an recommended image-based shopping service, Nexa automatically executes the service. The bottom shows an incorrect example: Bob takes a photo of a washing machine in a public laundry, but Nexa gives an incorrect recommendation. Bob exits the assistant and manually opens the laundry mini program. Nexa logs his GUI operations, extracts a macro script, adds it to the service list, and induces a rule for future recommendations.

Abstract

Smartphone services are growing rapidly in number and complexity, making traditional graphical user interfaces increasingly inefficient for service access. To improve accessibility, we propose Nexa, a unified, self-evolving context-aware assistant that integrates multi-modal user input (such as voice commands, camera input, screen content and motion gestures) and contextual information (such as time, location, network and notification) to recommend and execute smartphone services. For service recommendation, the assistant employs a dual-layered prediction module: a rule-based System 1 for fast matching, complemented by an LLM-based System 2 that performs reasoning when no rules apply. The system continuously learns from user interactions; when recommendations fail, it logs the user's subsequent GUI interactions and updates its prediction model by analyzing causal relationships among multimodal inputs,

context, and the final chosen service. Additionally, macro scripts are extracted from logged GUI traces and added to the service list. We implement a prototype on Android with a cloud backend. We assume that Nexa can reduce the effort required to access services and adapt to individual preferences over time, thereby enhancing user experience.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools.**

Keywords

Service Recommendation; Smartphone; Context-Aware Assistant; Large Language Model



This work is licensed under a Creative Commons Attribution 4.0 International License. *UbiComp Companion '25*, Espoo, Finland
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1477-1/2025/10
<https://doi.org/10.1145/3714394.3750594>

ACM Reference Format:

Li Chen, Kaige Yang, and Zixuan Gong. 2025. Nexa: A Context-Aware Assistant for Service Recommendation on Smartphones. In *Companion of the 2025 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp Companion '25)*, October 12–16, 2025, Espoo, Finland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3714394.3750594>

1 Introduction

As smartphone services continue to grow in number and complexity, interacting with them through graphical user interfaces (GUIs)—by manually navigating apps via icons and menus—has become increasingly cumbersome. In response, manufacturers have introduced various mechanisms to improve service access efficiency. Initially, pull-down search bars provided users with recently used apps and enabled app lookup through text input. Voice assistants were later introduced to support quick access to predefined services via speech. Context-aware systems such as Siri Suggestions¹ and Android’s negative-one screen² further enhanced accessibility by recommending services based on location and usage patterns. More recently, advances in interface understanding—enabled by large vision-language models—have led to new interaction paradigms, such as Google’s Circle to Search³ and Honor’s Magic Door⁴, where users can act directly on on-screen content by selecting visual elements. Despite these developments, service access remains fragmented across multiple entry points, each supporting different modalities and limited service sets. To address this, we aim to build a unified assistant that integrates diverse input modalities and provides access to the full range of services available on the smartphone through a single interaction interface.

Building such a unified assistant presents two key challenges. First, the space of smartphone services is open-ended and continuously expanding, making it infeasible to predefine a complete list of services. Second, user preferences vary significantly across individuals and contexts, making it difficult to rely on static rules or general language models for accurate intent prediction.

To address these challenges, we develop a self-evolving context-aware assistant that learns and adapts to user’s personal usage pattern through real-world interactions. Specifically, when activated, the assistant recommends services based on multimodal user input and context. If a recommended service is accepted, it is executed automatically. Otherwise, if the recommendation is incorrect, the user proceeds to access the intended service manually via the standard GUI. In such cases, the system logs the subsequent interaction trace, identifies the accessed service, and adds it to the service list as a macro script. It also analyzes the relationship between the service, the user’s input, and the context to generate new rules, thereby continuously updating its intent prediction model.

We implement a prototype of the assistant with an Android-based frontend and a cloud-based backend. The system adopts a dual-layered prediction architecture inspired by dual-process theory [3]: a rule-based System 1 for fast and efficient intent matching, and an LLM-based System 2 for deeper reasoning when predefined rules do not apply. The LLM component fulfills four key roles: (1) recommending services by inferring the user’s state from multimodal input and context, and interpreting intent in rule-missing scenarios; (2) generating follow-up questions to elicit missing parameters required for service execution; (3) constructing macro scripts by analyzing GUI interaction traces for previously unseen services; (4) inducing new rules by identifying causal relationships between services, user input, and context. This architecture allows

the assistant to continuously evolve by learning from user behavior and supporting both predefined and usage-derived services.

Our main contribution is the design of a self-evolving, context-aware assistant that serves as a unified entry point for accessing services on smartphones. The system leverages interaction data collected during users’ engagement with the assistant to support continuous adaptation, learning both user preferences and new services over time. Unlike always-on sensing methods commonly used in traditional data-driven machine learning approaches [6, 7], our assistant collects data only during explicit interactions, offering a more controllable and privacy-conscious alternative. This design improves practical deployability by balancing data utility and user trust. Furthermore, compared to traditional GUI-based access, the assistant introduces only a minimal interaction overhead—activating the assistant—while producing valuable data that can significantly enhance future service automation.

2 Related Work

2.1 Limitations of Existing Context-Aware Systems.

Current context-aware systems typically rely on data-driven machine learning approaches to model users’ contextual preferences [6, 7]. While effective in capturing personalized patterns, these methods face two major limitations. First, they require a substantial amount of user data collected over long-term periods, which makes them inefficient or even infeasible in scenarios where personalized contextual and behavioral data are sparse. Second, such systems often rely on passive, always-on sensing in the background, which raises privacy concerns and may hinder user adoption. These limitations highlight the need for self-evolving frameworks that can learn from sparser, more targeted interaction data while offering greater control over data collection.

2.2 Employments of LLMs

The widespread adoption of large language models (LLMs) has shown strong potential in enabling the four key components of our proposed framework. Firstly, for service recommendation, MAPLE [4] leverages LLMs to model contextual information for app prediction, which demonstrates that LLM-based embeddings offer more nuanced and precise representations of user behavior compared to traditional neural network models. Secondly, for parameter elicitation question generation, recent work [5] shows that LLMs can effectively generate follow-up questions to acquire missing parameters required for service execution. Thirdly, for macro script generation, LLMs have been used to extract executable macros from mobile GUI traces, as in [2], where both random and user-curated traces are annotated with natural language. Finally, for rule construction, LangAware [1] demonstrates how LLMs can semantically align high-level user instructions with machine-perceivable context features, reducing the gap between natural language and sensor-level execution. Collectively, these works suggest that LLMs are a promising foundation for building a context-aware assistant capable of handling out-of-coverage scenarios by reasoning over user input and context, and improving through reflection.

¹<https://support.apple.com/en-ca/guide/iphone/iph6f94af287/ios>

²<https://consumer.huawei.com/cn/support/content/zh-cn00896183/>

³<https://support.google.com/websearch/answer/145089577>

⁴<https://www.honor.com/cn/support/content/zh-cn15854511/>

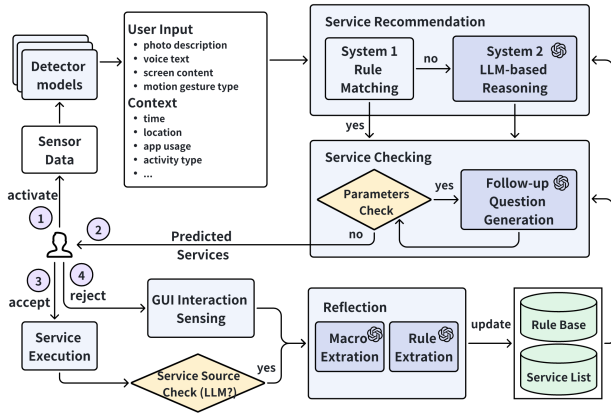


Figure 2: The framework of Nexa.

3 Nexa

3.1 Framework

Figure 2 illustrates the overall framework of Nexa. When activated, Nexa collects raw sensor data and processes it through a set of detector models to extract atomic user input and contextual features. These detector models fall into three categories: (1) lightweight data extractors that process signals from Android accessibility services and commercial APIs, such as time, location, app usage, and voice input; (2) machine learning models that interpret inertial sensor (IMU) data to infer activity states (e.g., walking, running) and motion gestures (e.g., shake, double tap); and (3) vision-language models that semantically parse visual input, including screen content and photo descriptions. The resulting multimodal signals are passed to a two-stage intent prediction module that follows a dual-process design: System 1 performs rule-based matching for fast prediction, while System 2 leverages LLM-based reasoning when no predefined rule applies.

If a candidate service is identified, Nexa proceeds to service checking. If required parameters are missing, the LLM generates a follow-up question to elicit the missing information from the user. Once all slots are filled, the predicted service is presented. If the user accepts the recommendation, the service is executed automatically. If the recommendation is rejected, the user proceeds to access the intended service manually via the GUI, during which Nexa activates a GUI interaction sensing module to log the interaction trace.

After the interaction, Nexa enters a reflection phase. It analyzes the collected GUI trace to extract a reusable macro script, which is added to the service list for future automation. In parallel, it examines the relationship between the original input, context, and the final service to generate new IF-THEN rules, which are integrated into the rule base. Additionally, services recommended by System 2 are also subject to reflection, allowing Nexa to gradually distill LLM-generated outputs into explicit rules for future predictions.

3.2 Rule Structure

We represent rules in an IF-THEN format, where the condition part may consist of one or more predicates combined using logical AND.

Table 1: The list of condition templates of rules.

	Type	Condition Template
User Input	Screen	Screen content contains <keywords> Screen content contains <object category>
	Photo	Photo contains <object category>
	Voice input	Voice command contains <keywords>
	Motion gesture	Motion gesture is <gesture>
Context	Time	At <time> Between <begin_time> and <end_time> Before <time> After <time>
		Leaving <location> <distance> Arriving at <location> <distance> At location <location> <distance> Not at location <location> <distance>
	Activity	Activity state is <activity> Started <activity> Ended <activity>
		Received message from <person> Receiving notification contains <keywords>
	Screen usage time	Screen usage time for this session is <minutes> minutes
	App	Opened <appName> Exited <appName> <appEvent> Currently using <appName> No record of app <appName> in past usage Used <appName> for <minutes> minutes today Single session usage of <appName> for <minutes> minutes
		Connected to <wifi> network Disconnected from <wifi> network
	Bluetooth	Connected to <bluetooth> device Disconnected from <bluetooth> device
		Started charging Ended charging When fully charged <Is/ Is not> charging Battery level is above <value> Battery level is below <value>
	Audio	<Is/ Is not> playing audio Microphone detected <keywords>

Table 1 summarizes commonly used condition templates, which we derive from existing automation tools (e.g., Apple Shortcuts⁵, Samsung Routines⁶, and Tasker⁷). The THEN part specifies the target service, defined by its intent, associated slots, and the app name (e.g., query_weather(location, date, app)).

3.3 Service Recommendation

Our service recommendation module adopts a hybrid rule+LLM architecture. For rule-based inference, when the conditions specified in a rule are satisfied, the corresponding service is recommended. For cases not covered by existing rules, the system invokes a LLM to perform reasoning and generate service predictions. The LLM prompt is structured with five key components: (1) **Prefix**: A brief statement of the assistant’s role and task objective. (2) **Reasoning steps**: Includes user state analysis, intent prediction, slot filling, and app selection. (3) **User input and context**: Structured as a JSON object derived from detector model outputs. (4) **Retrieved rules**: Contextual rules retrieved based on user state similarity. (5) **Service list**: Descriptions of both built-in and usage-derived services available in the system. Below is a simplified description of the prompt structure.

⁵<https://support.apple.com/zh-cn/guide/shortcuts/welcome/ios>

⁶<https://galaxy.store.samsung.com/detail/com.samsung.android.app.routineplus>

⁷<https://tasker.joaoapps.com/>

Prefix: You are an excellent mobile assistant, recommend services based on user input and context.

Reasoning Steps: The inference process is as follows:

- **User Status Analysis:** Analyze the context to deduce the user's current state, including their environment, physical activity, and phone usage.
- **Intent Recognition:** Infer the user's motivation for activating the assistant based on their status, and rank candidate services in `service_list`, each formatted as `<intent, slot, app>`.
- **Slot Filling:** Extract parameters from the user input and context to populate slots, represented as a dictionary with slot names as keys and specific values (e.g., date: "September 10, 2023", location: "Beijing").
- **App Selection:** Select the most suitable app from the user's installed apps.

User Input: `<user input and context>`.

Retrieved Rules: `<context-service pairs>`.

Service List: `<service-description pairs>`.

Output Format: Provide services as a `List<String>`.

3.4 Reflection

The reflection module consists of two components: **macro extraction** and **rule extraction**. The **macro extraction module** takes a sequence of GUI operations as input and generates a concise macro script by removing redundant steps through LLM-based reasoning. Each GUI operation is logged with detailed metadata, including the operation type (e.g., app open, click, swipe, text input), the corresponding layout, and a screenshot. The **rule extraction module** takes as input the user input, context, predicted service, condition templates, and the existing rule base. It generates new rules based on these inputs and updates the rule base accordingly.

4 Envisioned Evaluation and Dataset

We envision a two-part evaluation: (1) a system evaluation assessing two core modules—LLM-based service recommendation and rule extraction—using metrics such as top-1/top-k accuracy for service recommendation, and template/parameter extraction accuracy for rule extraction; and (2) a user study evaluating usability through measures including user satisfaction, perceived usefulness, and interaction overhead during real-world deployment.

As no public dataset directly supports evaluation of Nexa, we consider two strategies for dataset construction. The first strategy builds on interaction data collected during the user study. Each time the assistant is triggered, a context–service pair is recorded. Participants are then asked to confirm or annotate the corresponding rule, yielding labeled context–service–rule triples. These examples form a seed dataset, which can be further expanded through LLM-based data augmentation by varying context values and service parameters. The second strategy leverages the public IFTTT rule corpus⁸. Rules related to mobile services (e.g., SMS sending, app launching, WiFi control) are first filtered. For each rule, the LLM is prompted to generate plausible context values based on its trigger conditions. These are combined with the action (service) and original rule to form synthetic context–service–rule examples.

⁸https://www-users.cse.umn.edu/~fengqian/ifttt_measurement/

5 Discussion

Practical deployability. Nexa is designed for practical deployment by extending the existing smartphone voice assistants in both input and service coverage. On the input side, it expands beyond traditional voice and text to support multimodal inputs and contextual awareness. On the service side, it handles not only predefined services via APIs but also usage-derived services represented as macro scripts extracted from user interaction traces. However, the current framework is suited for services with fixed workflows, where all required parameters can be resolved prior to execution. Supporting services that require adaptive control based on runtime outcomes remains a challenge, calling for the integration of execution feedback into the assistant's interaction loop.

Accessibility and user empowerment. Nexa lowers the barrier to service access by reducing reliance on manual GUI navigation. This benefits a broad spectrum of users, including older adults with limited motor or cognitive abilities, as well as users in situationally constrained contexts (e.g., cooking, driving), where conventional interaction methods are impractical. In addition, Nexa enables users to define personalized triggers for service access by combining multiple input modalities and contextual signals. This is similar to iOS workflows that allow users to activate services by linking the Action Button with phone orientation⁹.

Digital intervention potential. The unified access paradigm supported by Nexa can also serve as a mechanism for intentional digital intervention. Unlike traditional app homepages, which frequently present personalized content to maximize engagement, Nexa offers a goal-oriented interaction model that is more aligned with users' actual service needs. This design has the potential to mitigate attention hijacking behaviors and reduce excessive screen time—a direction worth investigating through in-situ user studies.

Privacy and data collection. Nexa adopts an interaction-driven data collection strategy, in contrast to traditional always-on sensing methods commonly used in ubiquitous computing. This approach ensures that data is only collected during explicit user interactions, offering a more transparent and controllable privacy-preserving approach. However, since the current implementation relies on LLM-based reasoning, it introduces potential risks related to remote inference. Supporting on-device LLM inference could further enhance privacy and local autonomy for future deployment.

References

- [1] Weihao Chen, Chun Yu, Huadong Wang, Zheng Wang, Lichen Yang, Yukun Wang, Weinan Shi, and Yuanchun Shi. 2023. From Gap to Synergy: Enhancing Contextual Understanding through Human-Machine Collaboration in Personalized Systems. In *UIST 2023, San Francisco, CA, USA*. ACM. doi:10.1145/3586183.3606741
- [2] Forrest Huang, Gang Li, Tao Li, and Yang Li. 2024. Automatic Macro Mining from Interaction Traces at Scale. In *CHI 2024, Honolulu, HI, USA*.
- [3] Daniel Kahneman. 2011. *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- [4] Yonchanok Khaokaew, Hao Xue, and Flora D. Salim. 2024. MAPLE: Mobile App Prediction Leveraging Large Language Model Embeddings. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 8, 1 (2024), 10:1–10:25. doi:10.1145/3643514
- [5] Jianyu Liu, Yi Huang, Sheng Bi, Junlan Feng, and Guilin Qi. 2025. From Superficial to Deep: Integrating External Knowledge for Follow-up Question Generation Using Knowledge Graph and LLM. <https://arxiv.org/abs/2504.05801>
- [6] Abhinav Mehrotra, Robert Hendley, and Mirco Musolesi. 2016. PrefMiner: mining user's preferences for intelligent mobile notification management (*UbiComp '16*). ACM, New York, NY, USA, 12 pages. doi:10.1145/2971648.2971747
- [7] Vijay Srinivasan, Saeed Moghaddam, Abhishek Mukherji, Kiran K. Rachuri, Chenren Xu, and Emmanuel Munguia Tapia. 2014. MobileMiner: mining your frequent patterns on your phone. In *UbiComp '14, Seattle, WA, USA*.

⁹<https://support.apple.com/en-ca/guide/iphone/iphe89d61d66/ios>